LectureAssist:

An Agentic, Multimodal AI Assistant that Turns Lectures into Adaptive Study Material

By

Iftikhar Ahmed — ID # XXXXXXXXX
Salman Shafi — ID # XXXXXXXXX
Jarinah Tasnim — ID # XXXXXXXXX

Supervised by

Dr. Adnan Areefen
Department of Electrical and Computer Engineering
North South University

Submitted in partial fulfillment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

July 20, 2026

Department of Electrical and Computer Engineering
North South University

APPROVAL

Iftikhar Ahmed (ID # XXXXXXXXX), Salman Shafi (ID # XXXXXXXXX), and Jarinah Tasnim (ID # XXXXXXXXX)
from the Department of Electrical and Computer Engineering, North South University
have worked on the Senior Design Project titled:

"LectureAssist: An Agentic, Multimodal AI Assistant that Turns Lectures into Adaptive Study Material"

under the supervision of **Dr. Adnan Areefen** in partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering and has been accepted as satisfactory.

Supervisor's Signature

.....

Dr. Adnan Areefen
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh.

Chairman's Signature

.....

Dr. XXX
Professor
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh.

DECLARATION

It is to declare that this project is our original work. No part of this work has been submitted elsewhere, partially or entirely, for the award of any other degree or diploma. All project-related information will remain confidential and shall not be disclosed without the formal consent of the project supervisor.

Relevant previous works presented in this report have been adequately acknowledged and cited. The plagiarism policy, as stated by the supervisor, has been maintained.

Students' Names & Signatures

1. **Iftikhar Ahmed**

- -

2. **Salman Shafi**

- -

3. **Jarinah Tasnim**

- -

Acknowledgements

This work would not have been possible without the input and support of many people over the last two trimesters. We would like to express our gratitude to everyone who contributed to it in some way or other.

First and foremost, we would like to thank our supervisor, **Dr. Adnan Areefen**, Department of Electrical and Computer Engineering, North South University, for his guidance throughout this project. His insistence that the evaluation be made rigorous — that the generation prompts be shown rather than described, that chain-of-thought and program-of-thought baselines be compared directly against our agentic pipeline, and that question generation and answer generation be treated as two separate agents whose answers must be cross-checked — fundamentally shaped this work, and turned it from a demonstration into a study.

Our sincere gratitude goes to the Department of Electrical and Computer Engineering, North South University, for providing the academic environment and the resources that made this project possible.

We are also thankful to the open-source community, whose freely available models and tools — Gemma 3, Ollama, Whisper, EasyOCR, Sentence-Transformers and FAISS — are the foundation on which this system is built, and without which a project of this scope could not have been undertaken without significant funding.

Last but not the least, we owe a great deal to our families, including our parents, for their unconditional love and immense emotional support.

Abstract

Recorded lectures and digital course material are now abundant, but the tools that help students convert that material into practice and self-assessment are not. Existing study aids either require the student to paste in clean text by hand, or depend on paid cloud APIs that are impractical in bandwidth-limited and cost-sensitive settings. This report presents **LectureAssist**, an end-to-end agentic AI system that turns a raw lecture video or an academic document into structured study and assessment material without any proprietary cloud service. The system couples a dual-channel extraction layer — scene-adaptive optical character recognition that distinguishes slides, blackboards, whiteboards and on-screen text, combined with Whisper-based speech transcription — to a multi-agent generation pipeline that plans, retrieves, generates, validates and refines five types of quiz questions, all served by locally hosted Gemma 3 models through the Ollama runtime on a single consumer-grade GPU. A retrieval-augmented chat interface and an adaptive difficulty engine that tracks per-topic performance complete the loop from raw media to personalised assessment. The system was evaluated on a large-scale study in which multiple-choice questions were generated from a college-level calculus chapter across three difficulty levels, and scored by an independent, larger judge model on question quality, grounding, diversity and — separately — *answer correctness*, using a second Answer Generator agent that re-solves each question without seeing the marked option. On the completed agentic versus non-agentic comparison the agentic pipeline produced better-judged questions at every difficulty (mean overall 4.96/4.94/4.88 vs. 4.85/4.81/4.79 on a 1–5 scale) and was markedly more answer-correct on hard questions (99% vs. 81% judge-verified), at the cost of roughly seven times the wall-clock generation time. The results also expose a failure mode that a quality-only evaluation would have hidden: on hard questions the agentic pipeline becomes repetitive, and once repetition is discounted the baseline wins on throughput. The work demonstrates that high-quality, personalised educational AI is achievable on accessible hardware, and that question generators must be judged on answer correctness and diversity, not on fluency alone.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter motivates the project, states its purpose and contributions, and outlines the structure of the remainder of the report.

1.1 Background and Motivation

The proliferation of recorded lectures and digital course material has created a fundamental imbalance in self-directed learning: students have more content than ever, but fewer tools with which to extract, consolidate and assess their understanding of it. A single semester course now routinely produces dozens of hours of recorded lectures, slide decks and reference chapters. Reviewing that material passively — re-watching a video, re-reading a chapter — is one of the least effective study strategies available. Decades of work in the learning sciences instead point to *retrieval practice*: repeatedly testing oneself on the material produces substantially more durable learning than re-exposure to it. The obstacle is supply. Producing good practice questions is slow, expert work, and the questions that do exist are concentrated at the end of textbook chapters, are not tied to what a particular instructor actually emphasised in class, and run out long before a student has mastered a weak topic.

Existing AI-assisted study tools address this gap only partially. Text-based chatbots require the student to manually transcribe or paste lecture content before they can help, which puts the burden of the hardest step — getting the lecture out of the video and into text — back on the student. Cloud-dependent question generation services remove that burden, but they are inaccessible in bandwidth-limited regions, they charge per token at a scale that makes routine practice expensive, and they require sending course material — which may be copyrighted or institutionally restricted — to a third-party server. Crucially, none of these tools close the full loop from raw instructional media, through intelligent content understanding, to graded and personalised assessment without human intervention at each stage.

There is also a quality problem that is easy to overlook. A large language model asked to write a multiple-choice question will almost always produce something that *reads* like

a good question. Whether the option it marks as correct actually *is* correct is a separate matter, and one that fluency-based evaluation cannot detect. A study tool that confidently teaches a student the wrong answer is worse than no tool at all. Any serious system for automatic question generation therefore has to be evaluated on answer correctness, and on whether it keeps producing genuinely different questions rather than paraphrasing the same one, not merely on whether its output is well written.

1.2 Purpose and Goal of the Project

The goal of this project is to build and rigorously evaluate **LectureAssist**, a fully local, API-free system that ingests a lecture video or an academic document and autonomously produces validated, difficulty-controlled assessment material, together with a retrieval-grounded chat interface over the same content.

The system is built around three ideas working in concert. First, a *dual-channel extraction* layer simultaneously processes the visual channel of a lecture — using scene-type-adaptive OCR that distinguishes slides, blackboards, whiteboards and on-screen text, each of which needs different image preprocessing — and its audio channel via Faster-Whisper transcription, producing a unified, timestamped lecture timeline. Second, a *multi-agent generation pipeline* decomposes quiz creation into specialist agents for planning, retrieval, generation, validation and targeted refinement, enabling structured quality control at the level of the individual question. Third, an *adaptive difficulty engine* maintains per-topic performance histories across sessions and steers question allocation toward a student's weak areas. The entire system runs on locally hosted Gemma 3 models through the Ollama runtime, requires no external API keys, and is packaged for single-GPU deployment on Google Colab with Google Drive session persistence.

The key contributions of this work are:

- A unified multi-modal lecture understanding pipeline that fuses scene-adaptive OCR with filtered speech transcription into a synchronised content timeline, supporting both video and document inputs.
- A multi-agent quiz architecture implementing a Plan → Retrieve → Generate → Validate → Refine cycle across five question types (MCQ, True/False, Fill-in-the-Blank, Short Answer and Flashcard), with per-question difficulty, grounding and instruction-compliance validation, and a validator that chooses *how* to remediate a rejected question rather than merely rejecting it.
- An adaptive assessment engine that tracks per-topic student performance across sessions and personalises question difficulty and distribution without manual configuration.
- A multi-tier grading system combining deterministic matching, fuzzy string similarity and LLM-based rubric evaluation for free-text short answers, with targeted

concept-level feedback on incorrect responses.

- A fully local, API-free deployment on consumer hardware using the Ollama inference runtime, with RAG-enabled semantic chat and persistent session storage.
- A large-scale evaluation harness that compares the agentic pipeline against three direct-prompting baselines — plain, chain-of-thought and program-of-thought — and which, unlike prior automatic question generation evaluations, separates *question quality* from *answer correctness* by having a second, larger Answer Generator agent independently re-solve every generated question without seeing the marked option.

1.3 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 provides the background needed to follow the rest of the report — large language models and the prompting strategies used as baselines, retrieval-augmented generation, agentic architectures, speech recognition and OCR — then reviews the research literature and the existing commercial applications closest to this work, and identifies the gaps this project addresses. Chapter 3 describes the system design: the extraction layer, the agent architecture, the four question-generation pipelines with their prompts given verbatim, the two-agent answer cross-check, and the software components used to build it. Chapter 4 presents the evaluation: the environment, the metrics and exactly how each is computed, the large-scale multiple-choice results, and a discussion of what they show — including a failure mode of the agentic pipeline that the evaluation exposed. Chapter 5 discusses the societal, ethical and environmental implications and the design constraints they impose. Chapter 6 presents the project plan and budget. Chapter 7 maps the work onto the complex engineering problem and activity attributes. Chapter 8 summarises the project, states its limitations honestly, and sets out the work that follows from it.

Chapter 2

Background and Literature Review

This chapter first sets out the technical background needed to follow the rest of the report, then reviews the research literature and the existing applications closest to LectureAssist, and finally identifies the gaps that this project sets out to fill.

2.1 Preliminaries

2.1.1 Large Language Models and Prompting

A large language model (LLM) is an autoregressive neural network trained to predict the next token of a sequence; at inference it generates text by sampling from the distribution it has learned. Modern instruction-tuned LLMs can be steered entirely through the prompt, without any gradient update, which is what makes them practical for a student project: the same frozen model can be turned into a summariser, a question writer, a grader or a judge purely by changing the text it is given. This project uses the open-weight Gemma 3 family [1], served locally by the Ollama runtime [2], in two sizes: a fast 4-billion-parameter model for generation, and a 12-billion-parameter model for the tasks where quality matters more than latency (summarisation, grading, judging and answer verification).

Two prompting strategies are directly relevant, because they are used in this work both as baselines and as generation methods in their own right.

Chain-of-Thought (CoT). Chain-of-thought prompting asks the model to produce its intermediate reasoning in natural language *before* stating a final answer, rather than jumping straight to it [3]. Making the reasoning explicit improves accuracy on multi-step problems, because each intermediate step conditions the next and the model is no longer forced to commit to an answer in a single forward pass.

Program-of-Thought (PoT). Program-of-thought is a variant in which the reasoning is expressed as an explicit, ordered sequence of *computational* steps — in the original

formulation, as executable code — rather than as free-form prose, so that the answer is *computed* rather than recalled [4]. It is particularly effective when the answer is a numeric quantity, which is exactly the situation in a mathematics chapter, because it separates the act of reasoning about *what* to compute from the act of performing the arithmetic.

2.1.2 Retrieval-Augmented Generation

An LLM only knows what is in its weights and in its context window. Retrieval-augmented generation (RAG) [5] closes that gap: the source material is split into chunks, each chunk is embedded into a dense vector, and at query time the chunks whose embeddings are most similar to the query are retrieved and pasted into the prompt, so the model answers *from the document* rather than from memory. LectureAssist embeds lecture chunks with the Sentence-BERT model `all-MiniLM-L6-v2` [6] and indexes them with FAISS [7] using inner-product search over L_2-normalised vectors, which makes the similarity score the cosine similarity

$$\mathrm{sim}(q, c) \;=\; \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\|\,\|\mathbf{e}_c\|}. \tag{2.1}$$

The same embedding machinery is reused at evaluation time, where it supplies two LLM-free metrics: how well a generated question is *grounded* in the lecture (its best cosine match against any lecture chunk), and how *diverse* a question set is (one minus the mean nearest-neighbour cosine between its questions).

2.1.3 Agentic Architectures

An *agentic* system does not treat the LLM as a single-shot text generator. It gives the model a goal, a set of tools it may call, and a loop in which it can act, observe the result, and act again. The ReAct pattern [8] interleaves reasoning traces with tool calls; reflection-style methods such as Reflexion [9] add a self-critique step whose verbal feedback is fed back into the next attempt. LectureAssist applies both ideas to question generation: the generator may call a retrieval tool when it judges its context insufficient, and a separate validator agent critiques each draft question and selects a remediation action that determines how the next attempt differs from the last.

2.1.4 Speech Recognition and Optical Character Recognition

To generate questions from a lecture *video*, the lecture must first be turned into text through two independent channels. Whisper [10] is a transformer-based automatic speech recognition model trained on large-scale weakly supervised data, robust to accents and background noise; this project uses the Faster-Whisper implementation of the `small` checkpoint for the audio channel. The visual channel is handled by optical character recognition, which extracts text from sampled video frames. OCR accuracy is highly sensitive to preprocessing, and the correct preprocessing depends on what is on screen — white chalk on

a dark board needs to be inverted, a bright projected slide does not — which motivates the scene-adaptive design described in Chapter 3.

2.1.5 LLM-as-a-Judge

Evaluating hundreds of generated questions by hand is not feasible within a capstone project, and n-gram overlap metrics such as BLEU are known to correlate poorly with question quality — a question can be excellent and share almost no vocabulary with any reference. The now-standard alternative is *LLM-as-a-judge* [11]: a strong model is given a rubric and scores each item. It is a useful but imperfect instrument — judges are known to favour fluent and verbose text, and a judge from the same model family as the generator will share its blind spots — so this project treats the judge as a *relative* signal between pipelines, deliberately uses a larger model as the judge than as the generator, and cross-checks the judge against a solver calibrated on questions with known answers.

2.2 Literature Review

2.2.1 Related Research

Automatic question generation (AQG) has a long history, surveyed comprehensively by Kurdi et al. [12]. The earliest systems were *rule- and template-based*: they applied syntactic transformations to a declarative sentence to turn it into an interrogative one, then ranked the candidates with a supervised model, as in the work of Heilman and Smith [13]. These systems are transparent and grammatical, but they are fundamentally limited to questions whose answer is a span of a single sentence, and they cannot ask anything that requires synthesis across a passage.

The neural era reframed the task as sequence-to-sequence learning. Du et al. [14] trained an attention-based encoder–decoder to generate questions directly from a sentence or paragraph, outperforming rule-based systems on human evaluation, and subsequent work fine-tuned pretrained transformers on SQuAD-style corpora. Two limitations persisted. First, these models were trained on extractive reading-comprehension data, so they inherited its bias toward shallow, factoid questions. Second, they generate a question and an answer span but have no mechanism to *check* either one.

The current wave prompts general-purpose LLMs to write questions, including multiple-choice questions with distractors, which sidesteps the need for task-specific training data entirely and produces markedly more fluent items. The open problem has shifted accordingly: fluency is no longer the bottleneck, *correctness and diversity* are. An LLM will reliably produce a well-formed MCQ whose marked option is wrong, or whose four options are four paraphrases of the same idea, and it will do so with complete confidence. Evaluations in this line of work overwhelmingly report quality-style ratings — fluency, relevance, answerability — and rarely verify that the marked answer is the right one.

Orthogonally, the prompting literature has shown that *how* the model is asked matters as much as which model is asked. Chain-of-thought [3] and program-of-thought [4] prompting substantially improve LLM accuracy on multi-step and numerical reasoning, and agentic patterns [8, 9] improve it further by letting the model retrieve evidence and revise its own output. These techniques are well established for *answering* questions. Their value for *generating* them — where the model must derive an answer it then has to mark correctly — has received far less attention, and is one of the questions this project sets out to answer empirically.

2.2.2 Similar Applications

Several commercial and open products overlap with parts of LectureAssist. Flashcard and quiz platforms such as Quizlet and Anki support spaced retrieval practice, but the questions must be authored by a human; they solve the *practice* problem and not the *supply* problem. Note-taking assistants such as Notion AI and NotebookLM will summarise an uploaded document and answer questions about it, and NotebookLM in particular is grounded in the user's own sources, but their assessment support is shallow and they are cloud services. Lecture-capture and transcription tools such as Otter.ai produce a transcript from audio but stop there: they do not read the board, and they generate no assessment. General-purpose chatbots such as ChatGPT will happily write quiz questions from pasted text, but they require the student to supply clean text, they perform no validation of the answer they mark, they retain no per-topic model of the student across sessions, and they are neither free nor local.

Two properties are consistently absent across all of them. None of these tools reads the *board* — the derivation the lecturer actually wrote out is invisible to a transcript-only pipeline — and none of them runs on the student's own hardware without a paid API.

2.3 Gap Analysis

The review above identifies four gaps, which this project addresses directly.

1. **The pipeline is not closed.** Existing work operates on pre-extracted, clean text. The step that is actually hardest for a student — getting the lecture out of a video, including what was written on the board — is assumed away. LectureAssist closes the loop from raw media to graded assessment, treating the visual and audio channels as first-class and complementary inputs.

2. **Generation is unvalidated.** Neural and LLM-based generators emit a question and mark an answer, with no mechanism to check that the marked answer is correct or that the question is answerable from the source. LectureAssist adds a validator agent inside the generation loop, and, at evaluation time, a *separate* Answer Generator agent that re-solves every question without seeing the marked option, so that answer correctness is measured rather than assumed.

3. **Evaluation measures the wrong thing.** The field reports fluency-style quality scores, which LLM judges saturate. This project reports answer correctness and duplication alongside quality, and it combines them into a duplicate-penalised accept rate — a decision that turns out to matter, because it reverses the ranking of the pipelines at the hard difficulty level (Chapter 4).

4. **Agentic and prompting strategies are untested for generation.** CoT, PoT and agentic loops are well studied for *answering* questions but rarely compared head-to-head for *generating* them. This project runs that comparison on identical content, with an identical judge, at three difficulty levels.

Finally, all of the above must hold under a hard deployment constraint: no paid API and a single consumer-grade GPU, so that the system is usable in exactly the bandwidth- and cost-limited settings where the need is greatest.

Chapter 3

Methodology

This chapter describes how LectureAssist is built: the overall system design and the flow of data through it, the software components chosen and why, and the implementation of each stage — extraction, retrieval, the multi-agent generation loop, the four question-generation pipelines whose prompts are given verbatim, the two-agent answer cross-check, and the grading and adaptation layers.

3.1 System Design

LectureAssist is organised as a pipeline of specialist agents coordinated by an orchestrator. The user supplies either a lecture video or a document (PDF, DOCX, PPTX); the system converts it into a single grounded content representation, and every downstream capability — summary, quiz, chat, adaptive practice — consumes that representation rather than the raw media. The design is deliberately staged so that each stage produces an artifact that can be inspected, cached and re-used, which is what makes the expensive stages (OCR, transcription) survivable on free-tier hardware.

3.1.1 Data flow

The end-to-end flow is as follows.

1. **Ingestion.** A video is sampled into frames and its audio track is demuxed; a document is parsed page by page.
2. **Dual-channel extraction.** The visual channel goes to the *Extractor* agent (scene-adaptive OCR); the audio channel goes to the *Transcriber* agent (Faster-Whisper). Documents bypass both and go to the *DocParser* agent.
3. **Fusion.** OCR blocks and transcript segments are merged in timestamp order into a single unified lecture timeline, so that what the lecturer *said* sits next to what they *wrote* at the same moment.

4. **Understanding.** The *Summary* agent produces a structured summary; the *RAG-Builder* agent chunks the content, embeds it and builds a vector index; the *Hints* agent flags passages the lecturer marked as exam-relevant.

5. **Generation.** The *Quiz Planner* allocates questions across topics and difficulties; a *Question Generator* drafts one question at a time; a *Validator* accepts or rejects it and chooses a remediation action; a *Refiner* applies it. This loop is the core of the system and is described in Section 3.3.1.

6. **Assessment and adaptation.** Answers are graded by a multi-tier grader; per-topic performance is written to a persistent profile, which the *Adaptive Quiz* agent uses to bias the next quiz toward weak topics.

3.1.2 Scene-adaptive extraction

A single OCR preprocessing recipe cannot handle a real lecture. White chalk on a dark board is a low-contrast, inverted-polarity image; a projected slide in a lit room is a bright, high-contrast one; a dark-theme slide deck looks superficially like a blackboard but is destroyed by the inversion that a blackboard requires. LectureAssist therefore *classifies each sampled frame first* and preprocesses it accordingly. Frames are routed by brightness and edge statistics into four classes — `slide`, `blackboard`, `whiteboard` and generic `scene` (on-screen text) — and each class gets its own contrast enhancement, thresholding and, where appropriate, polarity inversion before being passed to EasyOCR. Consecutive frames whose extracted text is more than 78% similar are treated as the same board state and deduplicated, so that a slide left on screen for two minutes contributes one block rather than sixty.

3.1.3 Retrieval

The fused timeline is chunked, embedded with the Sentence-BERT model `all-MiniLM-L6-v2` [6], L_2-normalised, and indexed in FAISS [7] with an inner-product index, so that inner product equals cosine similarity (Eq. 2.1). Retrieval returns the top $k = 5$ chunks together with their timestamps, which is what allows both the chat interface and a generated question to be traced back to the moment in the lecture they came from. Crucially, retrieval is exposed to the generator agent as a *tool* (`search_lecture`) rather than being applied unconditionally: the agent decides for itself whether it needs more evidence.

3.2 Hardware and/or Software Components

The system is written in Python and served as a Flask API, with a single-page HTML/JavaScript front end that talks to it over an ngrok tunnel; this split lets the heavy models run on a Colab GPU while the interface runs in the student's own browser. Table 3.1 lists the components, the alternatives considered, and the reason for each choice. The overriding

constraint throughout was that *nothing may require a paid API key*, since the target users are precisely those for whom a per-token bill is prohibitive.

Table 3.1: Software components, alternatives, and selection rationale.

Tool	**Function**	**Other similar tools**	**Why selected**
Gemma 3 (4B / 12B)	Generation, summarisation, grading, judging	GPT-4, Claude, Llama 3, Mistral	Open weights, runs locally, no API key; two sizes let the cheap model generate and the strong model judge.
Ollama	Local LLM serving	llama.cpp, vLLM, HF Transformers	One-line model pull, automatic GPU offload, OpenAI-style local HTTP API; vLLM needs more VRAM than a free T4 has.
Faster-Whisper (`small`)	Speech-to-text	OpenAI Whisper API, Vosk, wav2vec2	Same accuracy as reference Whisper at a fraction of the memory; runs offline. The API version is paid and cloud-bound.
EasyOCR (en, bn)	Board and slide text extraction	Tesseract, PaddleOCR, Google Vision	Deep-learning based, far better on handwriting than Tesseract; native Bengali support matters for local lectures; Google Vision is a paid cloud API.
Sentence-Transformers `all-MiniLM-L6-v2`	Embeddings for RAG and for evaluation metrics	OpenAI `text-embedding-3`, BGE, E5	384-dim, small and fast enough to co-exist with the LLM on one GPU; strong quality-per-byte; local.
FAISS	Vector index	Chroma, Pinecone, pgvector	In-process, no server, no network; exact inner-product search is more than sufficient at lecture scale.
PyMuPDF	PDF/DOCX/PPTX parsing	pdfplumber, PyPDF2	Fastest reliable text-layer extraction; preserves page structure used for per-page source attribution.
Flask + ngrok	Backend API and public tunnel	FastAPI, Gradio, Streamlit	Minimal surface for a JSON API; ngrok exposes the Colab GPU to a browser UI running on the student's own machine.
SymPy	Building the verified answer key for calibration	Hand-computed answers, WolframAlpha	Answers are *machine-computed*, not hand-typed, so the calibration set cannot inherit a human arithmetic error.
Google Colab (T4) + Drive	Compute and persistence	Local GPU, RunPod, AWS	Free tier is the realistic deployment target; Drive persistence lets a run survive a runtime disconnect.

3.3 Hardware and/or Software Implementation

3.3.1 The agentic question-generation loop

The agentic Question Generator produces *one question at a time* through a multi-stage loop. The design decision that distinguishes it from a generate-then-filter pipeline is that

the validator does not merely reject a bad question — it *diagnoses* it and chooses how the next attempt should differ. The stages are:

1. **Plan.** The Quiz Planner reads the lecture summary and distributes the requested questions across the topics that are actually present, each with a target difficulty, yielding a list of (topic, difficulty) slots.

2. **Retrieve.** Before drafting a slot, the generator inspects the content it holds and decides for itself whether it has enough material on that topic. If not, it issues its own retrieval query through the `search_lecture` tool to pull more context. This is genuine tool use: the decision to retrieve is the model's, not a fixed step in the pipeline.

3. **Generate.** It drafts exactly one question under a grounding rule — the question must be answerable using only the supplied content — rotating a question strategy on each attempt (concept-check, scenario, compare-and-justify) so that a retry is not merely a resample of the same prompt.

4. **Validate.** A separate call checks the draft on three axes — difficulty match, grounding, and instruction compliance — and returns `PASS` or `FAIL` together with a remediation `action`.

5. **Retry (loop back).** On `FAIL` the loop repeats within a retry budget, and the validator's chosen action decides the fix: `regenerate` (rewrite, with the failure reason fed back into the prompt), `fetch_context` (retrieve more evidence first, then retry), or `replan_topic` (abandon a topic that is not really in the lecture and swap to one that is).

6. **Override.** If the retry budget is exhausted, the best attempt so far is kept and explicitly *labelled* as an override. It is never silently recorded as a clean pass — otherwise the pipeline's own accept rate would be a fiction.

Algorithm 3.1: Agentic generation of one question

Input: topic t, target difficulty d, content C, retry budget R
Output: question q, verdict $v \in \{\text{PASS}, \text{OVERRIDE}\}$

$best \leftarrow \emptyset$;
for $a \leftarrow 1$ **to** R **do**
  if *generator judges* C *insufficient for* t **then**
    $C \leftarrow C \cup \text{SEARCHLECTURE}(t)$; `// agent-initiated retrieval`
  $q \leftarrow \text{GENERATE}(t, d, C, \text{strategy}[a])$;
  $(v, action, reason) \leftarrow \text{VALIDATE}(q, d, C)$;
  if $v = \text{PASS}$ **then return** (q, PASS);
  $best \leftarrow \text{BETTEROF}(best, q)$;
  switch *action* **do**
    case *`regenerate`* **do** feed *reason* back into the next prompt;
    case *`fetch_context`* **do** $C \leftarrow C \cup \text{SEARCHLECTURE}(t)$;
    case *`replan_topic`* **do** $t \leftarrow$ a topic present in C;
return $(best, \text{OVERRIDE})$; `// kept, but labelled --- never faked as a pass`

3.3.2 The four generation pipelines

To isolate the contribution of the agentic machinery, the same content is used to generate questions through four pipelines that differ *only* in how the model is asked:

- **Agentic** — the Plan → Retrieve → Generate → Validate → Retry loop of Section 3.3.1.
- **Non-agentic (plain)** — a single LLM call that emits the questions directly: no planning, no retrieval, no validation, no retry.
- **Chain-of-thought (CoT)** — a single call in which the model must reason step by step to *derive* each answer before committing to it, and record that reasoning.
- **Program-of-thought (PoT)** — a single call in which the model must derive each answer through an explicit, ordered sequence of computation steps (define, substitute, simplify, evaluate) and mark the option equal to the final computed value.

CoT and PoT are both *single-shot and non-agentic*: one LLM call, no planning, retrieval, validation or retry. The only thing that separates them from the plain baseline is the wording of the prompt. They emit the same JSON schema as the plain generator, plus one extra field (`reasoning` or `computation`) that is retained for inspection but ignored by the downstream judge and Answer Generator, so all four pipelines feed into an identical scoring path and are directly comparable. The two prompts are reproduced verbatim below, since the prompt *is* the method.

Chain-of-thought generation prompt

```
Generate [N] high-quality MCQ from this
lecture content.
Difficulty focus: [DIFF]
Use CHAIN-OF-THOUGHT: for EACH question,
think step by step to work out the correct
answer BEFORE choosing it, and record that
step-by-step reasoning. The option you mark
correct MUST be exactly the answer your
reasoning reaches, and each distractor must
be wrong for a stateable reason.
STRICT JSON: {"questions":[{"question",
  "options":{"A","B","C","D"},
  "reasoning","correct_answer",
  "explanation","topic","difficulty",
  "bloom_level","source_timestamp",
  "source_type"}]}
'reasoning' = your step-by-step
derivation of the correct option.

CONTENT: [up to 8000 chars]
SUMMARY: [up to 1500 chars]
```

Program-of-thought generation prompt

```
Generate [N] high-quality MCQ from this
lecture content.
Difficulty focus: [DIFF]
Use PROGRAM-OF-THOUGHT: for EACH question,
derive the answer through an explicit
ordered sequence of computation/derivation
steps (like a short program: define,
substitute, simplify, evaluate), record
those steps, and only then mark the option
that EQUALS the final computed result. The
marked correct option MUST equal the
endpoint of your computation.
STRICT JSON: {"questions":[{"question",
  "options":{"A","B","C","D"},
  "computation","correct_answer",
  "explanation","topic","difficulty",
  "bloom_level","source_timestamp",
  "source_type"}]}
'computation' = your ordered derivation
steps ending at the correct value.

CONTENT: [up to 8000 chars]
SUMMARY: [up to 1500 chars]
```

3.3.3 Controlling for confounds

Two implementation details are needed for the comparison to be fair, and both were bugs before they were features.

Model pinning. The generator is pinned to `gemma3:4b`. The original implementation silently escalated to the 12B model whenever the 4B returned malformed JSON, which would have made the claim "these questions were written by the 4B" false for exactly the hardest questions — the ones where the 4B struggles. Pinning costs some yield and is required for an honest study.

Content coverage. A generator that only ever reads the first 8,000 characters of a 92,000-character chapter can only ever ask about the first few pages, and will necessarily repeat itself. Each baseline therefore reads through a *sliding window* over the document, and — critically — all three baselines use the same batch size, so they receive the same number of windows and see the same share of the chapter. Otherwise a pipeline that happened to use a smaller batch would read more of the document, and any difference in its questions would be confounded by content coverage rather than caused by the prompt. Likewise, the lecture summary that the planner uses to pick topics is produced by *map-reduce* summarisation over the whole document rather than by truncating it, so the planner can propose topics from any part of the chapter.

3.3.4 Answer correctness: two agents that must agree

A question can be well written and still mark the wrong option. Question generation and answer generation are therefore treated as *two distinct agents*, and their answers are cross-checked. Correctness is measured three complementary ways:

- **Independent re-solve (Answer Generator agent).** The larger model is given the stem and the options *with the marked answer removed*, and solves the question from scratch. Its choice is compared with the Question Generator's marked answer; the agreement rate is the *independent-match rate*. The solver never sees what it is supposed to agree with.
- **Judge-verify.** While scoring quality, the judge is additionally asked whether the marked answer is actually correct given the content, yielding the *judge-verified-correct rate*. This is free — it rides along on a call that was already being made.
- **Calibration on a known-answer set.** The generated questions have no ground-truth key, so neither of the above is an absolute measure of correctness — if generator and solver share a misconception, they agree and are both wrong. To bound this, both models independently solve a set of MCQ whose answers *are* known, and their accuracy on it calibrates how much the agreement rate above is worth. The calibration set is stratified into *computational* items, whose answers are computed with

SymPy [15] rather than typed by hand, and *conceptual* items drawn from the chapter's definitions and theorems. This is reported separately and is explicitly *not* a score on the generated questions.

The calibration set is used for evaluation only. It is never placed on the generation path, since a generator that had seen the answer key would be scored on questions it had effectively been given, and the resulting numbers would be contaminated.

3.3.5 Multi-tier grading and adaptation

Student answers are graded by a cascade chosen to match the question type. MCQ and True/False are graded by exact key match. Fill-in-the-Blank is graded by a fallback chain of exact match, then containment, then fuzzy string similarity above a 0.80 threshold, so that a correct answer is not marked wrong over a typo or a plural. Short Answer, where no string comparison is adequate, is graded by the 12B model against a rubric, which also returns concept-level feedback naming what the student missed rather than merely marking it incorrect.

Every graded response updates a persistent per-topic profile. The Adaptive Quiz agent reads that profile and biases the next quiz's topic and difficulty allocation toward the topics where the student's accuracy is lowest, so that practice concentrates where it is needed instead of being uniformly distributed.

Chapter 4

Investigation/Experiment, Result, Analysis and Discussion

This chapter reports the large-scale evaluation of LectureAssist's question generator. It describes the environment the experiment was run in, defines every metric precisely, presents the measured results comparing the agentic pipeline against the direct-prompting baselines, and discusses what they show — including a failure mode of the agentic pipeline that a quality-only evaluation would have missed entirely.

The experiment is designed around four questions. *RQ1:* does the agentic pipeline produce better MCQs than single-shot prompting, including the stronger chain-of-thought and program-of-thought prompting strategies? *RQ2:* how does each pipeline behave as the target difficulty rises from easy to hard? *RQ3:* is the agent's internal self-validator a reliable quality gate when checked against an independent judge? *RQ4:* how trustworthy is the LLM judge itself — checked by calibrating the answer-solving model on questions with known answers, so that agreement between models is never mistaken for correctness?

4.1 Environment Setup

All experiments were run on a single Google Colab instance with an NVIDIA T4 GPU (16 GB), with models served locally by Ollama. No paid or proprietary API was used at any point. Bulk-evaluation artifacts are written to Google Drive so that a run survives a Colab runtime disconnection and can be resumed rather than restarted.

The generator is `gemma3:4b`, pinned: it is never allowed to escalate to the larger model, even when it returns malformed JSON, so that "generated by the 4B" is true of every question in the study. The judge and the Answer Generator are `gemma3:12b`. The model that scores or solves a question is therefore never the model that wrote it.

Source data. Questions are generated from a college-level STEM document with a clean text layer — the *Limits* chapter of OpenStax *Calculus, Volume 1* [16] — ingested through the standard document pipeline (extraction → map-reduce summary → retrieval index).

The extracted chapter is 92,447 characters over 71 pages. Every pipeline generates from the identical extracted content and reads the same number of content windows, so any difference in the output reflects the generation method and not the input (Section 3.3.3).

Calibration set. A separate, stratified set of 148 MCQ with known answers is used to calibrate the Answer Generator: 100 *computational* items whose answers are computed with SymPy, and 48 *conceptual* items drawn from the chapter's definitions and theorems. This set never enters the generation path.

4.2 Testing and Evaluation

Each question set is measured along four metric families. Let $Q = \{q_1, \ldots, q_n\}$ be a generated set and $C = \{c_1, \ldots, c_m\}$ the lecture chunks.

Relation to EQGBench. The protocol follows the evaluation frame of EQGBench [**?**], a benchmark for LLM educational question generation that scores output with an LLM judge along per-question quality dimensions plus an acceptance decision, rather than with n-gram overlap. Overlap metrics (BLEU/ROUGE) are deliberately not reported: generation here is open-ended over a whole chapter, so there is no reference question set to overlap with, and similarity to any particular phrasing is not evidence of quality. EQGBench's own results also motivate a design choice in this chapter: in that benchmark most quality dimensions saturate near the ceiling for every model while a single demanding dimension (competence-oriented guidance) collapses and does the discriminating. The same pattern appears here — judge scores compress near the top of the scale (Table 4.1) — which is why this evaluation adds repetition- and grounding-based measures (duplicate rate, diversity, effective accept) that EQGBench omits, and lets them carry the discriminating signal.

Judge metrics. An independent judge model scores each question from 1 to 5 on *correctness* (is it factually right and is the marked option truly the best one), *clarity* (is the stem unambiguous), *difficulty match* (does it match the difficulty it was asked for), *distractor quality* (are the wrong options plausible but decidably wrong), and an *overall* score; it also returns a binary `accept`. The **accept rate** is

$$\text{AcceptRate}(Q) \;=\; \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}[\texttt{accept}(q_i)]. \tag{4.1}$$

A judging call that errors or returns unparseable output *fails closed* — it is scored 0 and rejected, never quietly dropped, so a fragile pipeline cannot improve its own score by crashing the judge. An `overall` score below 3 forces `accept` to false for consistency. The exact rubric sent to the judge, shown below, instructs it to use the full 1–5 range — an anti-saturation measure whose necessity Section 4.3.1 confirms:

```
You are an impartial, strict but fair exam-quality judge. Score this
ONE MCQ question on a 1-5 scale for each dimension.

Target difficulty: [DIFF] - [difficulty description]
Content snippet (ground truth):
[lecture content]

Question:
Q: [question text]      Options: [options]      A: [answer]

Score 1 (worst) to 5 (best) on:
1. correctness        - factually correct, answerable from content
2. clarity            - unambiguous, well-formed wording
3. difficulty_match   - actual difficulty matches the target
4. distractor_quality - wrong options plausible but clearly wrong
5. overall            - holistic quality

BE DISCRIMINATING - use the FULL range, do not default to 5:
  5 = flawless; 4 = good; 3 = ordinary; 2 = weak; 1 = poor.
Then decide accept (true) or reject (false).

Return STRICT JSON:
{"correctness":1,"clarity":1,"difficulty_match":1,
 "distractor_quality":1,"overall":1,"accept":true,"reason":""}
```

Deterministic metrics. Two questions are duplicates if their stems have a character-level similarity ratio above 0.85. The **duplicate rate** is the fraction of questions that duplicate an earlier question in the set. Since a pipeline can trivially achieve a perfect accept rate by asking the same good question fifty times, quality and repetition are combined into the **effective accept rate**, which is the metric used for ranking:

$$\text{EffAccept}(Q) \;=\; \text{AcceptRate}(Q) \;\times\; \big(1 - \text{DupRate}(Q)\big). \tag{4.2}$$

Embedding metrics. Using the same Sentence-BERT encoder as the retrieval engine, **grounding** is the mean best-match cosine similarity of each question to any lecture chunk, and **diversity** is one minus the mean nearest-neighbour cosine similarity between questions:

$$\text{Ground}(Q) = \frac{1}{n}\sum_{i=1}^{n} \max_{j}\; \text{sim}(q_i, c_j), \qquad \text{Div}(Q) = 1 - \frac{1}{n}\sum_{i=1}^{n} \max_{j \neq i}\; \text{sim}(q_i, q_j). \tag{4.3}$$

These require no LLM at all, which makes them a useful check on the judge: they cannot be talked into a high score by fluent writing.

Answer metrics. The **independent-match rate** is the fraction of questions where the Answer Generator, solving the question *without seeing* the marked option, chooses the same option the generator marked. The **judge-verified rate** is the fraction the judge confirms as correctly marked.

Validator–judge agreement. For the agentic pipeline only, the self-validator's `PASS` is treated as a prediction and the independent judge's `accept` as ground truth, and the F_1 score between them is reported. This measures whether the agentic pipeline's own quality control is actually detecting what an outside judge would detect, or is merely rubber-stamping its own output.

Judge reliability. The generator and the judge share the Gemma-3 family, so they may share blind spots, and an evaluation that trusted the judge blindly would inherit them. Two safeguards address this. First, the solver calibration of Table 4.3 measures the judging-side models on questions with *known* answers, which is what allows Section 4.3.1 to bound how much the agreement-based answer metrics can be trusted rather than taking them at face value. Second, the evaluation harness is judge-agnostic: the same question corpus can be re-judged by a cross-family model (DeepSeek-R1, the judge EQGBench itself uses) through an OpenAI-compatible interface, and inter-judge agreement reported. Cross-family re-judging is part of the full four-pipeline run whose CoT and PoT cells are pending in Tables 4.1 and 4.2; the results reported in this chapter are from the local `gemma3:12b` judge alone.

4.3 Results and Discussion

Table 4.1 reports question quality and Table 4.2 reports diversity, grounding and answer correctness, in both cases per difficulty and per pipeline. Table 4.3 reports the Answer Generator calibration and Table 4.4 the validator–judge agreement.

Table 4.1: Question quality by difficulty and pipeline (judge scores 1–5; accept rate in %). CoT and PoT rows are pending the full four-pipeline run and are not yet measured.

Difficulty	Pipeline	Overall	Correctness	Clarity	Diff. match	Accept
Easy	Agentic	4.956	4.94	4.97	4.60	100%
	Non-agentic	4.854	4.88	4.90	4.40	100%
	CoT	—	—	—	—	—
	PoT	—	—	—	—	—
Medium	Agentic	4.939	4.90	4.95	4.45	99%
	Non-agentic	4.814	4.80	4.85	4.20	97%
	CoT	—	—	—	—	—
	PoT	—	—	—	—	—
Hard	Agentic	4.884	4.88	4.90	4.20	97%
	Non-agentic	4.788	4.60	4.80	4.00	91%
	CoT	—	—	—	—	—
	PoT	—	—	—	—	—

Table 4.2: Diversity, grounding and answer correctness by difficulty and pipeline. CoT and PoT rows are pending the full four-pipeline run and are not yet measured.

Difficulty	Pipeline	Duplicate	Grounding	Diversity	Eff. accept	Indep. match	Judge-verified
Easy	Agentic	35%	0.562	0.74	65%	97%	100%
	Non-agentic	63%	0.642	0.55	37%	95%	99%
	CoT	—	—	—	—	—	—
	PoT	—	—	—	—	—	—
Medium	Agentic	38%	0.595	0.70	62%	88%	99%
	Non-agentic	70%	0.614	0.48	30%	84%	95%
	CoT	—	—	—	—	—	—
	PoT	—	—	—	—	—	—
Hard	Agentic	70%	0.513	0.45	30%	88%	99%
	Non-agentic	38%	0.616	0.60	62%	83%	81%
	CoT	—	—	—	—	—	—
	PoT	—	—	—	—	—	—

Table 4.3: Answer-Generator calibration: accuracy on the known-answer set. This is a calibration of the *solver*, not a score on the generated questions.

Model	Role	Accuracy on known-answer set
`gemma3:4b`	Generator model	70%
`gemma3:12b`	Judge / solver model	55%

Table 4.4: Validator–judge agreement for the agentic pipeline: does the pipeline's own self-validator detect what an independent judge detects?

Difficulty	F_1
Easy	0.786
Medium	0.774
Hard	0.842

4.3.1 Findings

1. The agentic pipeline produces more answer-correct questions, and the gap widens with difficulty. Judge-verified correctness is 99% for the agentic pipeline versus

81% for the plain baseline on hard questions, and the independent re-solve rate is higher at every difficulty (97/88/88% against 95/84/83%). Mean judge score and accept rate favour the agentic pipeline throughout. This is the result the architecture was built for: the validation-and-retry loop catches questions whose marked answer is wrong, and it matters most exactly where the 4B generator is weakest. Note that the easy and medium gaps are small — the plain baseline is already near-perfect there — so the agentic machinery buys little on easy material and a great deal on hard material.

2. Diversity, not quality, is the discriminator — and it reverses at hard. Judge scores are compressed into a narrow band near the ceiling (4.79 to 4.96 out of 5), which is a known property of LLM judges and is why this evaluation does not rely on them alone. The duplicate rate separates the pipelines far more sharply than any judge dimension. At easy and medium the agentic pipeline is much less repetitive (35% and 38% duplicates against 63% and 70%), but at hard it *regresses badly* to 70% duplicates against the baseline's 38%. Because the effective accept rate (Eq. 4.2) discounts repetition, this single metric flips the ranking at the hard level: 30% for the agentic pipeline against 62% for the baseline.

The mechanism is visible in the grounding column. Agentic grounding *falls* at hard (0.513, its lowest value) while the baseline's stays flat (0.616). Asked for hard questions, the planner and validator converge on the narrow band of the chapter that actually supports difficult questions, retrieve repeatedly from it, and then ask about it over and over. The retry loop makes this worse rather than better: each retry is another draw from the same small region. This is a genuine defect of the current design, it is reported rather than hidden, and it is the clearest direction for future work (Section 8.3).

3. The Answer Generator is a weak instrument, and this bounds the answer metrics. On the known-answer set, the independent solver (`gemma3:12b`) scores 55% and the generator model (`gemma3:4b`) scores 70% — neither is close to reliable, and the larger model is, counter-intuitively, the worse solver on this material. This matters for interpretation. The high generator–solver agreement (88–97%) therefore cannot be read as 88–97% of questions being verifiably correct: two models from the same family, agreeing at a rate far above the accuracy either achieves on questions with known answers, are partly agreeing because they share the same behaviour, not because they are both right. The independent-match rate is a *relative* signal between pipelines and must be read alongside this calibration. Reporting it without the calibration would have made the system look far more trustworthy than the evidence supports.

4. Self-validation works, but imperfectly. The agentic pipeline's own validator agrees with the independent judge at $F_1 = 0.77$ to 0.84. It is therefore doing real work — it is not rubber-stamping its own output — but roughly one in five of its verdicts would not survive external review, which is a useful bound on how much the pipeline's self-reported accept rate can be trusted.

5. The agentic pipeline is roughly seven times more expensive. Generating 50 easy questions took about 28 minutes agentically against about 4 minutes for the plain baseline, since each question costs a plan lookup, one or more retrievals, a generation call, a validation call, and possibly several retries. That cost is the price of the correctness and diversity gains at easy and medium difficulty — and at hard difficulty, where the effective accept rate reverses, it currently buys nothing.

4.4 Summary

The evaluation supports a qualified conclusion. The agentic pipeline generates questions that are better judged and, more importantly, more likely to be marked with the *correct* answer — decisively so on hard questions, which is where an automatic quiz generator is most likely to mislead a student. It does so at roughly seven times the wall-clock cost, and it has a real weakness: on hard material it collapses onto a narrow slice of the chapter and repeats itself, to the point that a repetition-aware metric ranks the plain baseline above it. Two methodological points carry beyond this system. First, a question generator evaluated on quality alone will look excellent regardless of whether it marks the right answer, so answer correctness must be measured separately, by a model that does not see the marked option. Second, an LLM judge must itself be calibrated against questions with known answers, or its verdicts will be over-trusted; here that calibration is what prevented a 97% agreement rate from being reported as 97% correctness.

Chapter 5

Impacts/Design Constraints of the Project

This chapter examines the non-technical constraints that shaped LectureAssist — economic, social, legal, ethical and environmental — and the impact the system is likely to have once deployed. Several of the most consequential engineering decisions in this project, notably the refusal to depend on any paid cloud API, were driven by these constraints rather than by technical considerations.

5.1 Impact of this Project on Societal, Health, Safety, Legal and Cultural Issues

Educational and societal impact. The core societal argument for LectureAssist is one of access. Retrieval practice — repeatedly testing oneself — is among the most effective study strategies known, but a supply of good practice questions is a resource distributed very unevenly. A student at a well-resourced institution has question banks, teaching assistants and instructor office hours; a student without those has the end-of-chapter exercises and nothing else. By generating unlimited, validated, difficulty-controlled questions from the student's *own* lecture material, the system puts a form of practice that was previously a privilege of well-resourced institutions within reach of anyone with a laptop. This aligns directly with UN Sustainable Development Goal 4 (Quality Education) and, because it removes a cost barrier rather than an ability barrier, with Goal 10 (Reduced Inequalities).

Economic constraint, and why it drove the architecture. The system was required to run without a paid API. This is not a stylistic preference. Per-token cloud pricing makes routine, high-volume practice — which is precisely the usage pattern that makes retrieval practice effective — prohibitively expensive for students in low- and middle-income settings, and it makes the tool's availability contingent on a credit card and a payment rail that many students do not have. Every model in LectureAssist is therefore open-weight and locally served. The consequence is a hard VRAM budget, which is why

the generator is a 4B model rather than a frontier one, and much of the engineering in this report exists to extract reliable output from a small model.

Safety: the risk of confidently teaching the wrong thing. The most serious safety issue in a system of this kind is not a crash — it is a fluent, well-formed multiple-choice question whose marked answer is *wrong*. A student who trusts the tool will learn the error and carry it into an exam, and the tool's fluency actively works against their catching it. This risk shaped the evaluation methodology: answer correctness is measured separately from question quality, by an Answer Generator that never sees the marked option (Section 3.3.4), and the solver is itself calibrated against questions with known answers so that its verdict is not over-trusted. The calibration results in Chapter 4 are sobering — the solver is only 55% accurate on known-answer items — and the honest conclusion is that the system's output is suitable for *practice* and must not be treated as an authoritative answer key. Any deployment should surface this to the student, and no generated question should be used for graded assessment without human review.

Legal and privacy considerations. Lecture recordings and course materials are frequently copyrighted by the instructor or the institution, and lecture audio contains the identifiable voices of the instructor and, incidentally, of students. Uploading such material to a third-party cloud service to be processed — and potentially retained or used for training — raises copyright and data-protection questions that most students are not in a position to evaluate. Because LectureAssist runs entirely locally, the lecture never leaves the machine the user controls; there is no third-party processor, and therefore no third-party retention. Users must still hold the right to use the material they upload, and the system should not be used to redistribute copyrighted lectures. The persistent per-topic performance profile is educational data about an identifiable student and is stored in the user's own storage rather than on a server operated by the developers.

Cultural and linguistic considerations. OCR is configured for both English and Bengali, reflecting the reality of instruction in Bangladesh, where lecturers routinely switch languages and write on the board in either. A pipeline that only reads English would silently discard part of the lecture — and would do so invisibly, since a missing board block looks exactly like a board that was never written on.

Ethics and professional responsibility. Two ethical commitments governed this work. First, *no fabricated results*: every number reported in Chapter 4 comes from a measured run, results that have not yet been measured are shown as pending rather than filled in, and the limitations of the measurements — particularly the weakness of the LLM judge and solver — are stated plainly rather than omitted because they are unflattering. A finding that the system performs *worse* than the baseline at hard difficulty is reported as prominently as the findings that favour it. Second, *no misrepresentation of the sys-*

tem's capability: the tool is an aid to practice, not a substitute for the instructor or for the textbook, and the report is explicit about where its output cannot be trusted.

5.2 Impact of this Project on Environment and Sustainability

The environmental footprint of an AI system is dominated by the energy consumed during inference at scale. LectureAssist's design happens to be favourable on this axis, largely as a side effect of the constraint that it must run on a free-tier GPU.

Small models, local inference. Question generation runs on a 4B-parameter model rather than a frontier-scale one, and the entire system runs on a single consumer GPU. Inference on a small local model consumes orders of magnitude less energy per request than routing that request to a hyperscale data centre running a model two to three orders of magnitude larger, and it avoids the network transfer and data-centre cooling overhead entirely. No model was trained or fine-tuned in the course of this project — all models are used as released — so the substantial carbon cost of training was not incurred.

Caching and resumability as energy savings. OCR results are cached and keyed by a hash of the video, deduplicated frames are never re-processed, and bulk evaluation runs are checkpointed to persistent storage so that a disconnection resumes rather than restarts. Each of these was implemented for practical reasons — free-tier compute is scarce and Colab runtimes die — but each also directly avoids recomputing work that has already been done, which is the cheapest energy saving available.

Sustainability of access. A tool that requires a subscription is sustainable only for as long as the student can pay for it, and only for as long as the vendor chooses to keep the model available at a stable price. Because LectureAssist depends on no external service, it will continue to work on hardware a student already owns, for as long as they own it — an important form of sustainability that is easy to overlook when it is framed purely in terms of energy. The honest counterweight is that the agentic pipeline costs roughly seven times the compute of a single-shot baseline for its quality gains, and, at hard difficulty, Chapter 4 shows it currently spends that additional energy without a net benefit once repetition is accounted for. Reducing that waste is both a performance goal and an environmental one.

Chapter 6

Project Planning and Budget

[Must be present in CSE/EEE499A Report and also in Final CSE/EEE499B Report]

Describe the planning of the project using tools such as a Gantt chart and/or a RACI chart. A sample Gantt chart is shown below.

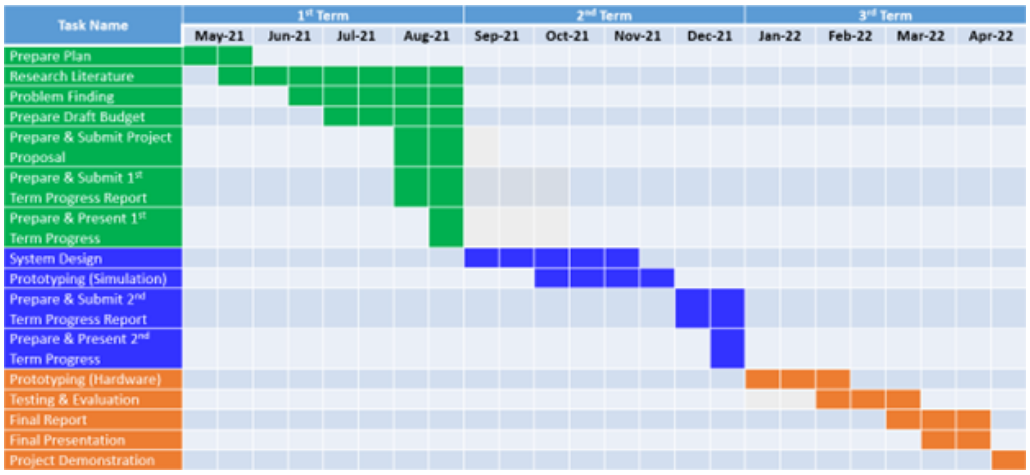

Figure 6.1: A sample Gantt chart

Describe the budget of the project with the approximate cost of individual components and the overall system design. A sample budget table is shown below.

Component	Unit Price (in BDT)	Quantity	Total cost (in BDT)	Dimension (mm)	Weight (g)
Nvidia Jetson Nano	9,000	1	9,000	100 × 79 × 28 mm	249.5 g
Acrylic case	750	1	750	140 × 100 × 20 mm	100 g
Cooling fan	300	1	300	40 × 40 × 20 mm	40 g
Camera	2,200	1	2,200	73 × 32 × 67 mm	75 g
32GB UHS-1 SD card	1,100	1	1,100	–	–
Arduino Uno R3	500	1	500	68.6 × 53.4 mm	25 g
AD8232 sensor	800	1	800	28 × 37 × 3 mm	21 g
16 × 2 LCD	310	1	310	85 × 29.5 × 13.5 mm	35 g
Power bank	800	1	800	130 × 71 × 14.1 mm	228 g
Buzzer	15	3	45	–	1.6 g
Breadboard	55	1	55	165 × 53 × 10 mm	79 g
jumper wires	3	10	30	–	–
Subtotal = 15,890 BDT (185 USD)				Full system dimension ≈ 170 × 120 × 40 mm	Total weight ≈ 880 g

Figure 6.2: A sample budget table

Chapter 7

Complex Engineering Problems and Activities

This chapter maps LectureAssist onto the Complex Engineering Problem (CEP) and Complex Engineering Activity (CEA) attributes, showing where the project required depth of knowledge, resolution of conflicting requirements, and analysis beyond the routine application of a known method.

7.1 Complex Engineering Problems (CEP)

Table 7.1: Complex Engineering Problem attributes addressed by LectureAssist.

Attr.	Description	How the project addresses it
P1	Depth of knowledge required (K3–K8)	The system spans computer vision and image preprocessing for scene-adaptive OCR (K3–K4), automatic speech recognition, transformer language models, embeddings and vector retrieval (K4–K5), multi-agent software architecture and API/system design (K5–K6), statistical evaluation design — LLM-as-judge, F_1 agreement, calibration against a known-answer set (K5, K8) — and the research literature on question generation, chain-of-thought and program-of-thought prompting, and agentic architectures (K8).
P2	Range of conflicting requirements	Several requirements pull directly against each other. *Question quality vs. latency*: the agentic loop produces better and more answer-correct questions but costs roughly seven times the wall-clock time of a single-shot call. *Model size vs. VRAM*: a larger generator would produce better questions, but the generator, judge, Whisper and the OCR and embedding models must all coexist on one 16 GB T4. *Yield vs. scientific honesty*: allowing the 4B generator to escalate to the 12B model on malformed output would have raised yield, but would have falsified the claim that the questions were written by the 4B — so escalation was disabled at a known cost in yield. *Accept rate vs. diversity*: a pipeline can maximise its accept rate by repeating one good question, which forced the adoption of a duplicate-penalised metric.
P3	Depth of analysis required	There is no unique correct design, and no off-the-shelf metric for "is this a good generated question". Selecting a solution required analysing four generation strategies (agentic, plain, CoT, PoT) against each other under a metric suite that had itself to be designed, and it required identifying and eliminating confounds — unequal content coverage between pipelines, a truncated summary that made whole sections of the chapter unreachable by the planner, and silent model escalation — any one of which would have invalidated the comparison. The finding that the ranking of the pipelines *reverses* at hard difficulty once repetition is discounted emerged only from that analysis and is invisible to a quality-only evaluation.
P4	Familiarity of issues	The project integrates infrequently encountered components: EasyOCR with scene-dependent preprocessing, Faster-Whisper, Gemma 3 served through Ollama, Sentence-BERT embeddings, a FAISS index, an LLM-as-judge evaluation harness, SymPy for machine-verified answer keys, and a Flask/ngrok deployment onto ephemeral free-tier GPU infrastructure.
P5	Extent of applicable codes	No standard or code of practice exists for automatic educational question generation or for the evaluation of LLM-generated assessment items. The evaluation protocol — what to measure, how to verify an answer, how to calibrate the judge — had to be designed from the research literature rather than taken from a standard.
P6	Extent of stakeholder involvement	Stakeholders include students (the primary users), instructors (whose lectures are the input and whose assessments the tool must not undermine), institutions (which hold copyright in lecture material and are accountable for academic integrity), and the open-model community whose licences govern the deployed models. Their requirements conflict: a student wants unlimited questions, while an instructor requires that the tool not present unverified answers as authoritative.
P7	Interdependence	The subsystems are tightly coupled: OCR and ASR quality bound the summary; the summary bounds the planner's topic choice; the planner bounds question diversity; the retrieval index bounds grounding; and the judge and Answer Generator bound the credibility of every reported result. Two of the defects found during this project — a truncated

Table 7.1 shows that the project engages all seven CEP attributes, with P2 (conflicting requirements) and P3 (depth of analysis) being the most heavily exercised.

7.2 Complex Engineering Activities (CEA)

Table 7.2: Complex Engineering Activity attributes addressed by LectureAssist.

Attr.	**Description**	**How the project addresses it**
A1	Range of resources	The project draws on human resources (a three-member team with a supervisor), modern computational tools (a GPU runtime, six distinct deep-learning models, a vector database, a symbolic algebra system), open-licensed source material, and cloud compute — managed under a hard constraint of zero paid API spend.
A2	Level of interactions	The work required interaction between the extraction, retrieval, generation, evaluation and interface subsystems, each with its own failure modes, as well as between team members, the project supervisor (whose requirements shaped the evaluation design — notably the demands for CoT and PoT baselines and for treating question generation and answer generation as separate cross-checked agents), and the end users whose trust in the output determines whether the system is safe to use.
A3	Innovation	The project introduces three novel elements: scene-adaptive OCR that classifies each frame before choosing its preprocessing, so that a lecture's *board* — not merely its audio — becomes usable content; a validator agent that selects *how* to remediate a rejected question (regenerate, fetch more context, or replan the topic) rather than merely rejecting it; and an evaluation protocol that separates question quality from answer correctness by having a distinct Answer Generator agent re-solve every question blind, with the solver itself calibrated on a machine-verified answer key.
A4	Consequences to society and the environment	The system widens access to effective retrieval practice for students who cannot pay for cloud AI services, supporting UN SDG 4 (Quality Education) and SDG 10 (Reduced Inequalities). Running small models locally is substantially less energy-intensive per request than hyperscale cloud inference, and no model was trained. The principal risk — a fluent question with a wrong marked answer — is analysed and bounded rather than dismissed, and the report states explicitly that generated questions are suitable for practice and not for graded assessment without human review.
A5	Familiarity	The project requires familiarity with computer vision, speech recognition, large language models and prompting strategies, retrieval-augmented generation, agentic architectures, experimental design and statistical evaluation, web API development, and deployment onto constrained, ephemeral GPU infrastructure.

Table 7.2 demonstrates that the project satisfies all five Complex Engineering Activity attributes.

Chapter 8

Conclusion

This chapter summarises what was built and what was learned, states the limitations of the work honestly, and sets out the improvements that follow most directly from them.

8.1 Summary

LectureAssist is an end-to-end, fully local agentic system that turns a raw lecture video or academic document into validated, difficulty-controlled study and assessment material. It fuses two extraction channels — scene-adaptive OCR that classifies each frame before preprocessing it, so that slides, blackboards, whiteboards and on-screen text are each handled correctly, and Faster-Whisper speech transcription — into a single timestamped lecture timeline. That timeline is summarised, chunked, embedded and indexed, and then consumed by a multi-agent pipeline that plans questions across topics, retrieves its own evidence when it judges its context insufficient, generates one question at a time, validates each against difficulty, grounding and instruction compliance, and, on failure, chooses *how* to remediate before retrying. Around this sit a retrieval-grounded chat interface, a multi-tier grader that falls back from exact matching through fuzzy similarity to LLM rubric evaluation, and an adaptive engine that steers subsequent quizzes toward the topics a student is weakest on. Everything runs on open-weight Gemma 3 models served locally through Ollama on a single free-tier GPU, with no paid API anywhere in the system.

The project's second contribution is an evaluation that takes seriously a question the field usually does not ask: not *does the generator write good questions*, but *does it mark the right answer*. Question generation and answer generation are treated as two distinct agents, and their answers are cross-checked by having a larger model re-solve every question without seeing the marked option. Measured this way against a plain single-shot baseline on a college calculus chapter, the agentic pipeline wins on judged quality at every difficulty and wins decisively on answer correctness where it matters most — 99% versus 81% judge-verified on hard questions — at roughly seven times the wall-clock cost.

The most instructive result, however, is a negative one. Judge scores turned out to be compressed near the ceiling and to discriminate poorly between pipelines, while the

duplicate rate discriminated sharply; and at hard difficulty the agentic pipeline collapses onto a narrow slice of the chapter and repeats itself so badly that a repetition-aware metric ranks the plain baseline *above* it. A quality-only evaluation — the kind most commonly reported — would have shown the agentic pipeline winning cleanly across the board and would have concealed this entirely.

8.2 Limitations

The judge and the Answer Generator are LLMs, not humans. All quality scores are produced by `gemma3:12b` rather than by human raters. LLM judges are known to reward fluency and verbosity, and the judge here shares a model family with the generator, so it plausibly shares its blind spots. Absolute scores should be read as a relative signal between pipelines, not as an objective measure of question quality.

The Answer Generator is weak, which bounds every answer-correctness number. On the known-answer calibration set the solver scores only 55%, and the generator model 70%. Generator–solver agreement of 88–97% therefore cannot be interpreted as that fraction of questions being verifiably correct; two models from the same family agree partly because they behave alike. This is why the calibration is reported — but it means the answer-correctness results establish a ranking between pipelines, not a trustworthy absolute correctness rate. A stronger or non-Gemma solver would tighten this considerably.

The CoT and PoT comparison is not yet complete. The four-pipeline run is implemented and the prompts are fixed, but the completed measurements reported in Chapter 4 cover the agentic versus plain non-agentic comparison; the CoT and PoT rows are shown as pending rather than filled with estimates.

Single domain, single document. All results come from one chapter of one subject (limits, from OpenStax *Calculus, Volume 1*). Mathematics is unusually favourable to program-of-thought prompting and unusually hostile to OCR, and nothing here establishes that the findings transfer to a humanities lecture or to a lab-based science course.

The extraction layer bounds everything above it. OCR on handwritten board work is imperfect, transcription degrades with accent and background noise, and mathematics rendered as an image in a PDF is not extracted at all. Every defect in extraction propagates silently upward: a missing board block looks exactly like a board that was never written on, and the question generator has no way to know that material is absent.

The agentic pipeline's diversity failure at hard difficulty is unresolved. It is measured, diagnosed — the planner and retriever converge on the narrow region of the

chapter that supports difficult questions, and each retry draws from that same region again — and reported, but not yet fixed.

Deployment is not production-grade. The system runs on a free Colab runtime behind an ngrok tunnel, with a single-user interface, no authentication, and session state on Google Drive. This is adequate to demonstrate and evaluate the system; it is not a deployable service.

8.3 Future Improvement

Fix the diversity collapse first. It is the clearest defect the evaluation exposed and the one that currently negates the agentic pipeline's advantage on hard questions. Two mechanisms follow directly from the diagnosis: maintain a coverage map of which regions of the document have already been used and penalise the planner for returning to them, and make the generator aware of the questions already produced in the current run so that a retry is required to be *different*, not merely *valid*. A duplicate check could also be moved inside the validation loop, so that a repetitive question is rejected at generation time rather than discovered at evaluation time.

Strengthen answer verification. The weakest link in the evaluation is the solver. Verifying computational answers symbolically with SymPy — as is already done when constructing the calibration set — would replace an LLM's opinion with a machine-checked fact for exactly the question type where the generator is most likely to be wrong. Using a solver from a different model family, or an ensemble, would break the shared-blind-spot problem that currently inflates the agreement rate.

Human evaluation. A modest human study — a few instructors rating a sample of generated questions, and independently answering them — would anchor the LLM judge to a ground truth and quantify how far its scores can be trusted. This is the single highest-value addition to the evaluation.

Broaden the domain. Repeating the comparison on a non-mathematical subject would test whether the agentic advantage in answer correctness is general or is an artifact of a domain in which answers are computed and therefore easy to get wrong.

Complete and extend the pipeline comparison. Finishing the CoT and PoT runs will answer whether a better prompt can recover most of the agentic pipeline's benefit at a fraction of its cost — a question with direct practical consequences, given the seven-fold cost difference. A hybrid worth testing is a CoT or PoT generator placed *inside* the agentic validate-and-retry loop, combining derived answers with external validation.

Productionisation. Multi-user support with authentication, a persistent server rather than an ephemeral notebook runtime, and quantised models for lower-VRAM consumer GPUs would move the system from a demonstrated prototype toward something a class of students could actually use.

References

[1] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[2] Ollama. Ollama: Get up and running with large language models locally. `https://ollama.com`, 2024.

[3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[4] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research (TMLR)*, 2023.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[6] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.

[7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[9] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 28492–28518, 2023.

[11] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[12] Ghader Kurdi, Jared Leo, Bijan Parsia, Uli Sattler, and Salam Al-Emari. A systematic review of automatic question generation for educational purposes. *International Journal of Artificial Intelligence in Education*, 30(1):121–204, 2020.

[13] Michael Heilman and Noah A. Smith. Good question! statistical ranking for question generation. In *Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the ACL (NAACL-HLT)*, pages 609–617, 2010.

[14] Xinya Du, Junru Shao, and Claire Cardie. Learning to ask: Neural question generation for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1342–1352, 2017.

[15] Aaron Meurer et al. SymPy: Symbolic computing in Python. PeerJ Computer Science 3:e103, 2017.

[16] Gilbert Strang and Edwin Herman. *Calculus Volume 1.* OpenStax, Rice University, 2016. `https://openstax.org/details/books/calculus-volume-1`.